

Implementation Reflection

AI Customer Support Agent | Amazon Bedrock AgentCore & Strands SDK

■ Design Decision: MCP & Gateway Architecture

Integrating backend tools via the Model Context Protocol (MCP) through the AgentCore Gateway was chosen over hardcoding direct API integrations into the agent's core logic. By leveraging the `streamable_http_client` and `MCPClient`, the agent treats complex operations—such as order tracking and refund processing—as standardized, discoverable tool calls. This decoupled architecture abstracts the underlying AWS Lambda infrastructure, ensuring that backend e-commerce updates do not force a redeployment of the `BedrockAgentCoreApp`. It also establishes a unified security boundary where the Gateway handles authentication and routing, keeping the primary agent logic clean and strictly focused on conversational state management.

■ Challenge Encountered: State Synchronization

A significant hurdle involved synchronizing the execution flow between cross-session AgentCore Memory and the Bedrock Knowledge Base (RAG) pipeline. During early testing, the agent failed to accurately answer policy questions that depended on historical user context (e.g., "Given my loyalty tier, how many days do I have to return this?"). The initial implementation executed the knowledge base search before the agent had fully loaded the user's profile. To resolve this, the `MemoryHook` was reconfigured to execute immediately prior to the agent's evaluation step. This guaranteed that customer preferences, names, and past session states were fully populated in the context window before the `search_knowledge_base` tool invoked the Retrieve API, resulting in highly grounded, status-specific responses.

■ Production Consideration: Financial Sandbox Limits

Securing financial transactions and code execution environments is critical before live deployment. While the AgentCore Code Interpreter provides an isolated, secure sandbox for calculating precise loyalty discounts, directly piping these programmatic outputs into the refund-processing Lambda introduces unacceptable business risk. In a production environment, hallucinated inputs or edge-case mathematical logic could trigger unauthorized payouts. To safely deploy, a strict validation layer must be implemented entirely outside the agent's control. This requires enforcing hard-coded maximum financial thresholds within the Lambda function itself and instituting a mandatory human-in-the-loop (HITL) approval queue for any refund exceeding automated safety limits.